



MEDIACREST
TRAINING COLLEGE

Machine Learning II: Classification Foundations & Ensemble Learning Models

Week 5, Day 3: Transitioning to Classification, Ensemble
Learning Models

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

12th September 2026

Abstract

This presentation advances our classification journey by introducing robust ensemble learning models to predict Atherosclerotic Heart Disease (AHD).

- **Problem Formulation:** Defining clinical objectives, target variables, and real-world implications of false positives/negatives.
- **Data Pipeline:** Robust preprocessing strategies for mixed data types (numeric and categorical) using `scikit-learn` Pipelines.
- **Ensemble Models:** Training and tuning Decision Trees, Random Forests, Extra Trees, AdaBoost, Gradient Boosting, and XGBoost.
- **Evaluation:** Rigorous hold-out test set evaluation using ROC-AUC, PR-AUC, and Confusion Matrices to select the optimal clinical model.

Session Objectives

- Formulate a clinical machine learning problem with clear success metrics.
- Build leak-proof preprocessing pipelines for mixed data types.
- Implement stratified train/test splits to maintain class distributions.
- Train and tune six distinct ensemble classifiers using `RandomizedSearchCV`.
- Evaluate models on a hold-out test set using clinical-grade metrics (Recall, F1, ROC-AUC, PR-AUC).
- Justify the final model selection based on the trade-off between sensitivity and precision.

Part 1: Problem Formulation

"Defining the clinical objective and real-world impact."

Clinical Objective & Target Variable

Predicting Atherosclerotic Heart Disease (AHD)

Enable early intervention and risk stratification for patients in county health facilities.

- **Target Variable:** HD (Binary: 0 = No Disease, 1 = Disease Present).
- **Key Predictors:** Age, MaxHR, Oldpeak, ChestPain type, and Ca/Thal values.
- **Hypothesis:** Tree-based ensemble models (e.g., XGBoost, Random Forest) will outperform single classifiers due to their ability to capture non-linear interactions and handle mixed data types effectively.

Real-World Implications

False Negatives (Type II Error)

Missed cardiac risk.

Impact: Delayed treatment, potential acute events (MI, stroke).

Cost: HIGH. Recall/Sensitivity must be prioritized.

False Positives (Type I Error)

Unnecessary angiography/stress tests.

Impact: Patient anxiety, wasted healthcare resources.

Cost: MODERATE. Precision balances this.

Part 2: Data Importation & Exploration

"Understanding the data before building the pipeline."

Dataset Overview & Dependencies

- **Dataset:** Heart.csv containing 303 patients and 15 variables.
- **Target variable:** HD, representing heart disease status.
- **Categorical variables:** ChestPain and Thal.
- The analysis is implemented entirely in Python.

Python Libraries

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import (
7     train_test_split, RandomizedSearchCV, StratifiedKFold
8 )
9 from sklearn.pipeline import Pipeline
10 from sklearn.compose import ColumnTransformer
11
```

Loading the Data

- The dataset is loaded using pandas.
- We inspect the structure, variable types, and target distribution.
- Missing-value representations are handled during import.

Python Implementation

```
1 # Visualization settings
2 sns.set_theme(style="whitegrid"), plt.rcParams['figure.figsize'] = (10, 6)
3 # Load the Heart Disease dataset
4 df = pd.read_csv('Heart.csv', na_values=['NA', '?', 'NaN', ''])
5 # Inspect dataset structure
6 print(df.info())
7 # Examine target-class distribution
8 print(df['HD'].value_counts(normalize=True))
9
```

Key Variables

ChestPain, Thal: Categorical | HD: Binary Target

Variable Definitions

Variable	Definition	Clinical Relevance
Age	Patient's age in years	Increasing age is associated with higher CVD risk
ChestPain	Type of chest pain experienced	Chest-pain characteristics provide important diagnostic information
RestBP	Resting blood pressure (mmHg)	Elevated blood pressure is a major cardiovascular risk factor
Chol	Serum cholesterol level (mg/dL)	Elevated cholesterol is associated with atherosclerotic disease
MaxHR	Maximum heart rate achieved during exercise	Reduced exercise capacity may indicate impaired cardiac function
Oldpeak	ST-segment depression induced by exercise	Greater ST depression may indicate myocardial ischemia
Ca	Number of major vessels identified by fluoroscopy (0–3)	A higher number of affected vessels indicates greater disease burden
Thal	Thallium stress-test result	Fixed or reversible defects may indicate myocardial perfusion abnormalities
HD	Heart disease diagnosis; binary target (0/1)	Primary outcome used for model classification

Missing Observations & Imputation Strategy

- Initial audit reveals minimal missingness: Ca (4 missing) and Thal (2 missing).
- **Strategy:** Handle missingness within the preprocessing pipeline to prevent data leakage.

Python Implementation

```
1 # Check missing values
2 missing = df.isnull().sum().sort_values(ascending=False)
3
4 print(missing.head(3))
5
6 # Output:
7 # Ca      4
8 # Thal    2
9 # Age     0
10
```

[Pro Tip] Imputation within a Pipeline ensures that statistics such as the median are learned **only from the training data**, preventing information from the test set from entering the model.

Target Variable Distribution

- The dataset is relatively balanced, which is favorable for binary classification.
- **Class Distribution:** approximately 54% No Disease (0) and 46% Disease (1).

Python Implementation

```
1 # Check target distribution
2 print(df['HD'].value_counts(normalize=True))
3
4 # Output:
5 # 0      0.541322
6 # 1      0.458678
7
8 # Visualize class distribution
9 sns.countplot(x='HD', data=df)
10
11 plt.title('Distribution of Heart Disease (HD)')
12 plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
13 plt.show()
14
```

Part 3: Data Preprocessing & ML Pipeline Creation

“Building robust, leak-proof preprocessing workflows.”

The ML Pipeline Concept

Why Use Pipelines?

- **Prevents Data Leakage:** Preprocessing steps are fitted **only on training data** during cross-validation.
- **Reproducibility:** Bundles transformers and the final estimator into one cohesive object.
- **Cleaner Code:** Eliminates repeated manual transformations of `X_train` and `X_test`.

Key Principle

Fit preprocessing on training data — never on the full dataset.

Numeric Pipeline

- **Features:** Age, RestBP, Chol, MaxHR, Oldpeak, Ca.
- **Steps:** Median imputation followed by standardization to mean = 0 and variance = 1.

Python Implementation

```
1 from sklearn.impute import SimpleImputer
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import Pipeline
4
5 # Define numerical features
6 numeric_features = [
7     'Age', 'RestBP', 'Chol',
8     'MaxHR', 'Oldpeak', 'Ca'
9 ]
10
11 # Build numerical preprocessing pipeline
12 numeric_transformer = Pipeline(steps=[
13     ('imputer', SimpleImputer(strategy='median')),
14     ('scaler', StandardScaler())
15 ])
16
```

Categorical Pipeline

- **Features:** Sex, ChestPain, Fbs, RestECG, ExAng, Slope, Thal.
- **Steps:** Most-frequent imputation followed by ordinal encoding with explicit handling of unknown categories.

Python Implementation

```
1 from sklearn.preprocessing import OrdinalEncoder
2
3 # Define categorical features
4 categorical_features = [
5     'Sex', 'ChestPain', 'Fbs', 'RestECG',
6     'ExAng', 'Slope', 'Thal'
7 ]
8
9 # Build categorical preprocessing pipeline
10 categorical_transformer = Pipeline(steps=[
11     ('imputer', SimpleImputer(strategy='most_frequent')),
12     ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value',
13     unknown_value=-1
14     ))
15 ])
```

Creating the Preprocessor

- Numeric and categorical pipelines are combined using ColumnTransformer.
- Each variable is automatically routed to its appropriate preprocessing workflow.

Python Implementation

```
1 from sklearn.compose import ColumnTransformer
2
3 # Combine numerical and categorical pipelines
4 preprocessor = ColumnTransformer(
5     transformers=[
6         ('num', numeric_transformer, numeric_features),
7         ('cat', categorical_transformer, categorical_features)
8     ]
9 )
10
```

Key Principle

One preprocessor, two specialized transformation workflows.

Part 4: Data Partitioning

“Ensuring representative training and testing sets.”

Train / Test Split Strategy

- **Goal:** Evaluate the model's ability to generalize to completely unseen data.
- **Split:** 80% training and 20% testing.
- **Stratification:** stratify=y preserves the proportion of heart disease cases in both sets.

Python Implementation

```
1 SEED = 42
2
3 # Remove redundant target variable AHD
4 # Keep HD as the classification target
5 X = df.drop(columns=['AHD', 'HD'])
6 y = df['HD']
7
8 # Stratified 80/20 train-test split
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y,
11     test_size=0.20,
12     random_state=SEED,
13     stratify=y
14 )
15
```

Class Distribution Check

- We verify that stratification preserved the target-class proportions after the split.

Python Implementation

```
1 # Training-set size and class distribution
2 print(f"Training Set: {X_train.shape[0]} | "
3       f"Class Dist: {y_train.value_counts().to_dict()}")
4 # Output: Training Set: 242 | Class Dist: {0: 131, 1: 111}
5 # Testing-set size and class distribution
6 print(f"Testing Set: {X_test.shape[0]} | "
7       f"Class Dist: {y_test.value_counts().to_dict()}")
8 # Output: Testing Set: 61 | Class Dist: {0: 33, 1: 28}
9 # Verify training-set proportions
10 print(y_train.value_counts(normalize=True))
11
12 # Output: 0 = 0.5413, 1 = 0.4587
13
```

Verification

Training and testing sets retain approximately the same class proportions as the original dataset.

Part 5: Model Training & Hyperparameter Tuning

“Optimizing ensemble models for peak performance.”

Ensemble Learning Models Overview

Bagging (Parallel)

- **Random Forest:** Combines multiple decorrelated trees to reduce variance.
- **Extra Trees:** Randomizes both samples and split points to improve robustness.

Boosting (Sequential)

- **AdaBoost:** Sequentially focuses on previously misclassified observations.
- **Gradient Boosting:** Fits new models to the residual errors of previous models.
- **XGBoost:** Optimized boosting with regularization to control overfitting.

Key Idea

Ensemble learning combines multiple weak or base learners to produce a stronger predictive model.

Model Pipelines: Tree-Based

- Random Forest and Extra Trees are implemented within the same preprocessing framework.
- The preprocessing step is therefore applied consistently before model fitting.

Python Implementation

```
1 from sklearn.ensemble import ( RandomForestClassifier, ExtraTreesClassifier
2 )
3 from sklearn.tree import DecisionTreeClassifier
4
5 # Random Forest Pipeline
6 rf_pipeline = Pipeline(steps=[
7     ('preprocessor', preprocessor),
8     ('rf', RandomForestClassifier(random_state=SEED, n_jobs=-1,      class_weight='
9         balanced'))])
10
11 # Extra Trees Pipeline
12 extra_trees_pipeline = Pipeline(steps=[
13     ('preprocessor', preprocessor),
14     ('extra_trees', ExtraTreesClassifier(
15         random_state=SEED, n_jobs=-1, class_weight='balanced'))])
```

Model Pipelines: Boosting & XGBoost

- Boosting models are constructed using the same preprocessing pipeline.
- XGBoost is configured with logloss evaluation and parallel processing.

Python Implementation

```
1 from sklearn.ensemble import (AdaBoostClassifier, GradientBoostingClassifier)
2 import xgboost as xgb
3
4 # Gradient Boosting Pipeline
5 gb_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
6                               ('gb', GradientBoostingClassifier(random_state=SEED))])
7
8 # XGBoost Pipeline
9 xgb_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
10                               ('xgb', xgb.XGBClassifier(random_state=SEED, eval_metric='logloss',
11                                                         n_jobs=-1))])
12
```

Hyperparameter Tuning: RandomizedSearchCV

- RandomizedSearchCV evaluates 30 randomly selected parameter combinations.
- **Cross-Validation:** 5-Fold Stratified CV.
- **Scoring:** ROC-AUC, measuring the model's discrimination ability.

Python Implementation

```
1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
2
3 search = RandomizedSearchCV(estimator=pipeline, param_distributions=params,
4                             n_iter=30, cv=cv, scoring='roc_auc', n_jobs=-1, random_state=SEED, refit=True)
5
6 search.fit(X_train, y_train)
7
```

Tuning Strategy

Search for strong hyperparameter combinations while preserving class proportions across folds.

Hyperparameter Grids: Decision Tree & RF

Python Implementation

```
1 # Decision Tree parameter distribution
2 param_dist_dt = {'dt__max_depth':[None, 3, 5, 7, 10, 15, 20],
3                  'dt__min_samples_split':[2, 5, 10, 15],
4                  'dt__criterion':['gini', 'entropy', 'log_loss'],
5                  'dt__class_weight':['balanced', None]}
6
7 # Random Forest parameter distribution
8 param_dist_rf = {'rf__n_estimators':[100, 200, 300, 400, 500],
9                  'rf__max_depth':[None, 5, 10, 15, 20, 25],
10                  'rf__min_samples_split':[2, 5, 10, 15],
11                  'rf__max_features':['sqrt', 'log2', None],
12                  'rf__class_weight':['balanced', 'balanced_subsample']}
13
```

Parameters Explored

Tree depth		Number of trees		Split criteria		Class weighting
------------	--	-----------------	--	----------------	--	-----------------

Hyperparameter Grids: Extra Trees & AdaBoost

Python Implementation

```
1 # Extra Trees parameter distribution
2 param_dist_extra_trees = {'extra_trees__n_estimators':[100, 200, 300, 400, 500],
3     'extra_trees__max_depth':[None, 5, 10, 15, 20, 25],
4     'extra_trees__min_samples_split':[2, 5, 10, 15],
5     'extra_trees__max_features':['sqrt', 'log2', None],
6     'extra_trees__class_weight':['balanced', 'balanced_subsample']}
7 }
8
9 # AdaBoost parameter distribution
10 param_dist_ada = {'ada__n_estimators':[50, 100, 150, 200, 300, 400],
11     'ada__learning_rate':[0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 1.5]}
12 }
13
```

Tuning Focus

Model complexity, learning rate, number of estimators, and feature randomness.

Hyperparameter Grids: GB & XGBoost

Python Implementation

```
1 # Gradient Boosting parameter distribution
2 param_dist_gb = {'gb__n_estimators':[50, 100, 150, 200, 300, 400],
3                  'gb__learning_rate':[0.01, 0.03, 0.05, 0.1, 0.15, 0.2],
4                  'gb__max_depth':[2, 3, 4, 5, 6],
5                  'gb__subsample':[0.7, 0.8, 0.9, 1.0]
6 }
7
8 # XGBoost parameter distribution
9 param_dist_xgb = {'xgb__n_estimators': [100, 200, 300, 400, 500],
10                  'xgb__max_depth': [2, 3, 4, 5, 6],
11                  'xgb__learning_rate':[0.01, 0.03, 0.05, 0.1, 0.2],
12                  'xgb__subsample':[0.7, 0.8, 0.9, 1.0],
13                  'xgb__colsample_bytree':[0.7, 0.8, 0.9, 1.0],
14                  'xgb__reg_alpha':[0, 0.01, 0.1, 1],
15                  'xgb__reg_lambda':[0.1, 1, 5, 10]}
```

Ensemble Pipeline Configuration

- All six ensemble pipelines are evaluated using the same cross-validation and ROC-AUC scoring framework.

Python Implementation

```
1 ensemble_pipelines = [  
2     ('Decision Tree', dt_pipeline, param_dist_dt),  
3     ('Random Forest', rf_pipeline, param_dist_rf),  
4     ('Extra Trees', extra_trees_pipeline,  
5         param_dist_extra_trees),  
6     ('AdaBoost', ada_pipeline, param_dist_ada),  
7     ('Gradient Boosting', gb_pipeline, param_dist_gb),  
8     ('Extreme Gradient Boosting', xgb_pipeline,  
9         param_dist_xgb)  
10 ]  
11  
12 results_ensemble = {}  
13
```

Executing RandomizedSearchCV

- The best fitted search object is retained for each model.

Python Implementation

```
1 for name, pipeline, params in ensemble_pipelines:
2
3     search = RandomizedSearchCV(
4         pipeline, params,
5         n_iter=30, cv=cv,
6         scoring='roc_auc',
7         n_jobs=-1,
8         random_state=SEED,
9         refit=True
10    )
11
12    search.fit(X_train, y_train)
13    results_ensemble[name] = search
14
15    print(f"{name}: "
16          f"Best ROC-AUC = {search.best_score_:.4f}")
17
```

Cross-Validation Results (ROC-AUC Ranking)

Rank	Model	Best CV ROC-AUC	Training Time (s)
1	Extra Trees	0.9067	36.37
2	Extreme Gradient Boosting	0.9006	5.56
3	Gradient Boosting	0.8983	12.86
4	Random Forest	0.8976	53.09
5	AdaBoost	0.8958	23.48
6	Decision Tree	0.8572	12.11

Observation

Ensemble approaches generally outperformed the single Decision Tree. **Extra Trees achieved the highest cross-validated ROC-AUC (0.9067).**

Interpretation of CV Results

- **Extra Trees:** Highest CV ROC-AUC (0.9067), indicating strong discrimination during validation.
- **XGBoost:** Second highest ROC-AUC (0.9006) with substantially lower training time (5.56 s).
- **Random Forest:** Strong and stable performance with a CV ROC-AUC of 0.8976.
- **Decision Tree:** Lowest CV ROC-AUC (0.8572), suggesting weaker generalization than the ensembles.
- **Clinical consideration:** CV ROC-AUC alone is insufficient. The independent test set should emphasize **Recall (Sensitivity)** and **False Negative Rate (FNR)** to minimize missed disease cases.

Next Step

Evaluate all tuned models on the independent hold-out test set.

Part 6: Hold-Out Test Set Evaluation

"Proving generalization on unseen clinical data."

Hold-Out Test Set Evaluation Setup

- We extract the best estimator from each RandomizedSearchCV object.
- We generate predictions and probabilities on the unseen `X_test`.

Python Implementation

```
1 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
2     f1_score, roc_auc_score, confusion_matrix)
3 eval_metrics = {}
4 for name, search in results_ensemble.items():
5     model = search.best_estimator_
6     y_pred = model.predict(X_test)
7     y_prob = model.predict_proba(X_test)[: , 1]
8     cm = confusion_matrix(y_test, y_pred)
9     tn, fp, fn, tp = cm.ravel()
10    metrics = {'Accuracy': accuracy_score(y_test, y_pred),
11              'Precision': precision_score(y_test, y_pred, zero_division=0),
12              'Recall': recall_score(y_test, y_pred, zero_division=0),
13              'F1-Score': f1_score(y_test, y_pred, zero_division=0),
14              'ROC-AUC': roc_auc_score(y_test, y_prob),
15              'FNR': fn / (fn + tp) if (fn + tp) > 0 else 0,
16              'y_prob': y_prob, 'model': model, 'confusion_matrix': cm}
17    eval_metrics[name] = metrics
```

Hold-Out Test Set Results: Metrics Table

Model	Acc	Prec	Rec	F1	ROC-AUC	PR-AUC
Random Forest	0.9016	0.8667	0.9286	0.8966	0.9740	0.9687
Extra Trees	0.8852	0.8182	0.9643	0.8852	0.9632	0.9482
XGBoost	0.8852	0.8889	0.8571	0.8727	0.9426	0.9343
Gradient Boosting	0.8525	0.9130	0.7500	0.8235	0.9578	0.9456
AdaBoost	0.8525	0.8800	0.7857	0.8302	0.9405	0.9284
Decision Tree	0.7869	0.7586	0.7857	0.7719	0.8523	0.8558

Key Finding

Random Forest achieved the highest Accuracy, ROC-AUC, PR-AUC, and F1-Score. **Extra Trees** achieved the highest Recall (Sensitivity).

Hold-Out Test Set Results: Confusion Matrices

Random Forest (Balanced)

- **TN:** 29 | **FP:** 4
- **FN:** 2 | **TP:** 26
- *FNR:* 7.14%

Excellent balance. Only 2 missed cases.

Extra Trees (Sensitivity-First)

- **TN:** 27 | **FP:** 6
- **FN:** 1 | **TP:** 27
- *FNR:* 3.57%

Lowest FNR. Only 1 missed case, but 2 extra false alarms.

Hold-Out Test Set Results: ROC & PR Curves

- Visualizing the trade-off between True Positive Rate and False Positive Rate.

Python Implementation

```
1 plt.figure(figsize=(10, 8))
2 for name, values in eval_metrics.items():
3     fpr, tpr, _ = roc_curve(y_test, values['y_prob'])
4     auc_score = roc_auc_score(y_test, values['y_prob'])
5     plt.plot(fpr, tpr, linewidth=2, label=f"{name} (AUC = {auc_score:.4f})")
6 plt.plot([0, 1], [0, 1], linestyle='--', linewidth=1.5, label='No Skill (AUC = 0.5000)')
7
8 plt.xlabel('False Positive Rate (1 - Specificity)')
9 plt.ylabel('True Positive Rate (Sensitivity)')
10 plt.title('ROC Curves: Six Ensemble Models - Hold-Out Test Set')
11 plt.legend(loc='lower right')
12 plt.grid(alpha=0.9)
13 plt.show()
```

Model Selection: The Clinical Trade-off

- **Random Forest:** The strongest **overall** performer. Highest Accuracy (90.16%), ROC-AUC (0.9740), and F1-Score (0.8966). It provides the best balance between identifying disease and avoiding unnecessary tests.
- **Extra Trees:** The strongest **sensitivity-oriented** alternative. Highest Recall (96.43%) and lowest False Negative Rate (3.57%).
- **The Trade-off:** Extra Trees achieves this sensitivity at the cost of lower Precision (81.82%), meaning more healthy patients will be subjected to unnecessary, costly angiography.

Clinical Directive

If the primary mandate from the Ministry of Health is "**miss no cases at all costs**", choose Extra Trees. If the mandate is "**optimal resource allocation with high reliability**", choose Random Forest.

Recommended Final Model

Final Recommendation: Random Forest

For this study, **Random Forest** is selected as the overall final model.

- **Justification:** It achieved the highest hold-out ROC-AUC (0.9740) and PR-AUC (0.9687), indicating superior discriminative power.
- Its Recall of 92.86% is still exceptionally high for a clinical setting, successfully identifying 26 out of 28 actual disease cases.
- It avoids the severe precision drop seen in Extra Trees, ensuring the model remains trustworthy and cost-effective for county health facilities.

Part 7: Deliverables & Next Steps

"Consolidating today's learning."

Deliverable: Ensemble Model Evaluation Report

Evening Lab Submission Requirement

Participants must submit a concise Jupyter Notebook or PDF report containing:

- 1 **Pipeline Summary:** Code for the `ColumnTransformer` and final chosen model pipeline.
- 2 **Tuning Results:** Table of Best CV ROC-AUC and training times for all 6 models.
- 3 **Hold-Out Evaluation:** Confusion Matrix, Classification Report, and ROC/PR curves for the **Random Forest** model.
- 4 **Policy Memo:** A 1-paragraph justification for selecting Random Forest over Extra Trees, explicitly referencing the Precision/Recall trade-off and clinical costs (FN vs FP).

Key Takeaways & Thank You!

- **Pipelines are non-negotiable:** They prevent data leakage during imputation and scaling.
- **Ensembles beat single trees:** Random Forest, Extra Trees, and XGBoost consistently outperformed the baseline Decision Tree.
- **CV $\neq$ Final Truth:** Cross-validation guides tuning, but the hold-out test set dictates the final clinical decision.
- **Context dictates metrics:** In healthcare, Recall (Sensitivity) is paramount, but must be balanced against Precision to avoid resource exhaustion.

Victor Wandera Lumumba

Statistician / Data Analyst / ML Expert

Mediacrest Training College | Nairobi, Kenya

Email: lumumbavictor172@gmail.com